

Manual técnico — AsistentelA

Proyecto final — Diseño de Software para Equipos Móviles (VIII semestre)

Tecnologías: .NET MAUI, C#, XAML, MVVM (CommunityToolkit.Mvvm), SQLite (sqlite-net-pcl), API REST (Pollinations, compatible con Chat Completions).

1. Objetivo del sistema

AsistentelA es una aplicación móvil multiplataforma (Windows/Android) que permite a un estudiante enviar consultas académicas a un modelo de IA a través de una API REST pública, visualizar la respuesta, conservar un historial local en SQLite y administrar de forma segura la clave de acceso al servicio.

2. Arquitectura general

La aplicación sigue el patrón **MVVM (Model-View-ViewModel)** con inyección de dependencias nativa de .NET MAUI.

```
Views (XAML) <--> ViewModels (CommunityToolkit.Mvvm) <-->
Services <--> API REST / SQLite
```

Carpeta	Responsabilidad	Clases principales
Models	Representar JSON de la API y entidades locales	ChatRequest , ChatMessage , ChatResponse , ChatChoice , ConversationRecord

Carpeta	Responsabilidad	Clases principales
Services	Acceso a la API, base de datos y almacenamiento seguro	IService , DatabaseService , ApiService
ViewModels	Estado, comandos y reglas de presentación	ChatViewModel , HistoryViewModel , DetailViewModel , SettingsViewModel
Views	Interfaz XAML y navegación	HomePage , ChatPage , HistoryPage , DetailPage , SettingsPage
Raíz	Arranque y navegación global	MauiProgram.cs , AppShell.xaml

Todas las dependencias (HttpClient , servicios, ViewModels y páginas) se registran en MauiProgram.cs mediante el contenedor de DI de .NET, y se inyectan por constructor en cada página y ViewModel.

3. Modelo de datos

3.1 Modelos de la API (Models/ChatModels.cs)

Representan el contrato JSON del endpoint POST /v1/chat/completions :

```
{
  "model": "openai/gpt-5.4-nano",
  "messages": [
    { "role": "system", "content": "..." },
    { "role": "user", "content": "..." }
  ]
}
```

La respuesta se deserializa en

```
ChatResponse.Choices[0].Message.Content .
```

3.2 Entidad local (`Models/ConversationRecord.cs`)

Tabla SQLite `ConversationRecord` :

Campo	Tipo	Descripción
<code>Id</code>	<code>int</code> (PK, autoincrement)	Identificador único
<code>Prompt</code>	<code>string</code>	Consulta enviada
<code>Answer</code>	<code>string</code>	Respuesta recibida
<code>CreatedAt</code>	<code>DateTime</code>	Fecha/hora UTC de creación
<code>Category</code>	<code>string</code>	Categoría asignada por el usuario
<code>IsVerified</code>	<code>bool</code>	Indica si el usuario verificó la respuesta

4. Servicios

4.1 `ApiKeyService`

Guarda y recupera la clave de API usando `SecureStorage` (cifrado nativo de la plataforma). Expone `SaveAsync` , `GetAsync` , `Remove` y `HasKeyAsync` . La clave **nunca** se escribe en código, archivos de configuración, logs ni capturas de pantalla (RF08).

4.2 `AiService`

Encapsula el consumo de la API REST:

- Valida longitud del prompt (5–2000 caracteres) antes de llamar a la red (RF02).
- Construye el `HttpRequestMessage` con encabezado `Authorization: Bearer {clave}` .

- Usa `HttpClient` inyectado (con `BaseAddress` y `Timeout` configurados en `MauiProgram.cs`).
- Soporta cancelación mediante `CancellationToken` (RF04).
- Traduce errores en excepciones específicas: `ArgumentException` (validación), `InvalidOperationException` (clave faltante), `HttpRequestException` (error del servidor) y `OperationCanceledException` (cancelación/timeout).
- Expone `TestConnectionAsync` para el botón "Probar conexión" de Configuración.

4.3 DatabaseService

Encapsula `SQLiteAsyncConnection` sobre un archivo en `FileSystem.AppDataDirectory` . Implementa `SaveAsync` , `GetAllAsync` , `GetByIdAsync` , `UpdateAsync` y `DeleteAsync` sobre `ConversationRecord` (operaciones CRUD del historial, RF05–RF06).

5. ViewModels y comandos

ViewModel	Propiedades observables clave	Comandos ([RelayCommand])
ChatViewModel	Prompt , Answer , StatusMessage , IsBusy , SelectedModel , Category , PromptTemplates	AskCommand , CancelCommand , SaveCommand
HistoryViewModel	Records (ObservableCollection), SearchText	LoadCommand , OpenDetailCommand , DeleteCommand

ViewModel	Propiedades observables clave	Comandos ([RelayCommand])
DetailViewModel	Record , recibe Id por query parameter Shell	ToggleVerifiedCommand
SettingsViewModel	ApiKeyInput , SelectedModel , HasKey	SaveKeyCommand , RemoveKeyCommand , TestConnectionCommand

DetailViewModel usa [QueryProperty(nameof(Id), "Id")] para recibir el identificador del registro al navegar desde HistoryPage mediante `Shell.Current.GoToAsync(nameof(DetailPage), new Dictionary<string, object> { { "Id", record.Id } })`.

6. Navegación (Shell)

AppShell.xaml define un TabBar con cuatro rutas fijas (HomePage , ChatPage , HistoryPage , SettingsPage) y registra en el code-behind la ruta de DetailPage , que no aparece en las pestañas y se alcanza únicamente por navegación programática desde el historial.

7. Seguridad, privacidad y uso responsable

Riesgo	Mitigación implementada
Exposición de credenciales	SecureStorage + campo IsPassword="True" ; la clave nunca se registra ni se muestra en pantalla tras guardarse.
Respuestas incorrectas o inventadas	Mensaje de aviso tras cada respuesta y mecanismo de verificación manual (IsVerified) en el Detalle.

Riesgo	Mitigación implementada
Datos personales en el prompt	Aviso visible en Configuración y en Inicio; validación de longitud del prompt.
Dependencia de un proveedor externo	El identificador del modelo es configurable (<code>ApiKeyService.AvailableModels</code>) y el <code>BaseAddress</code> del <code>HttpClient</code> está centralizado en <code>MauiProgram.cs</code> .
Errores de red/servidor	Manejo explícito de <code>HttpRequestException</code> , <code>OperationCanceledException</code> y excepciones de validación, con mensajes claros para el usuario.

8. Cómo compilar y ejecutar

1. Requisitos: Visual Studio Community con la carga de trabajo ".NET Multi-platform App UI development".
2. Abrir la solución `AsistenteIA` y restaurar paquetes NuGet (`CommunityToolkit.Mvvm` , `sqlite-net-pcl`).
3. Seleccionar el destino (Windows Machine o un emulador/dispositivo Android).
4. Ejecutar (F5). En el primer arranque, ir a **Configuración** e ingresar una clave de API válida antes de usar **Consultar IA**.

9. Extender el proyecto

- **Nuevo proveedor de IA:** cambiar `BaseAddress` en `MauiProgram.cs` y ajustar `ChatRequest` / `ChatResponse` si el formato JSON difiere.
- **Nuevo campo en el historial:** agregar la propiedad en `ConversationRecord` ; SQLite la incorporará al recrear la tabla o mediante una migración manual (`ALTER TABLE`).
- **Nuevas plantillas de prompt:** agregar cadenas a `ChatViewModel.PromptTemplates` .

10. Trazabilidad con los requisitos funcionales (RF01–RF10)

Requisito	Dónde se implementa
RF01	ChatPage.xaml (Editor de prompt)
RF02	AIService.AskAsync (validación de longitud)
RF03	AIService.AskAsync (POST a la API)
RF04	ChatViewModel (IsBusy , StatusMessage , manejo de excepciones)
RF05	ChatViewModel.SaveCommand + DatabaseService.SaveAsync
RF06	HistoryViewModel (filtro, eliminar) + DetailPage
RF07	ChatViewModel.PromptTemplates + Picker en ChatPage.xaml
RF08	ApiKeyService + SettingsPage (Entry IsPassword="True")
RF09	Sección "Acerca de" en SettingsPage.xaml
RF10	Proyecto .NET MAUI ejecutable en Windows y Android